

TDS200 · Kryssplattform

Session 3

State & Local Persistence

Approaches Comparison · AsyncStorage · SecureStore · Hooks

Cross-Platform Mobile Development · Lecturer: Yuan Lin

What you'll learn today

- Theory: 5 main mobile app approaches
- A new layout with only Tab and Stack navigation
- Manage state with `useState` and side-effects with `useEffect`
- Persist data on the device: `AsyncStorage` vs `SecureStore`
- Re-run logic when a screen refocuses (`useFocusEffect`)



Project step

A post feed from dummy data, saved locally so it survives restarts.

How a 2-hour session runs

0:00-0:05	Recap & objectives Where we are in the project; today's goals
0:05-0:45	Theory, State, effects & pull-to-refresh Five cross-platform approaches, Hooks, rendering lists efficiently
0:45-1:00	Break —
1:00-1:40	Screen refocus & Local storage (live) Persist & reload the feed
1:40-1:45	Assignment brief & Q&A This week's deliverable, then questions

Five Main Mobile App Approaches

Approach	How it works	Examples	Main strengths	Main limitations
Native	Build directly for one platform using platform-specific languages and SDKs	Swift/SwiftUI for iOS, Kotlin/Jetpack compose for Android	Best performance, full platform access, best user experience	Separate codebase for iOS and Android
Hybrid (WebView)	Web app wrapped inside a native container using WebView	Ionic, Cordova, Capacitor	One codebase, fast development, web skills reusable	Performance and native feel can be weaker
PWA	Web application enhanced to behave like an app through browser features	React/Vue/Angular PWA	Easy deployment, no app store required, works across devices	Limited native API access, weaker iOS support
Interpreted	Shared JavaScript code runs through a native bridge or runtime and renders native UI	React Native, Expo	Cross-platform, native-like UI, large ecosystem	Native modules and performance tuning may be needed
Cross-compiled	Code is compiled ahead of time into native machine code or native UI components	Flutter, .NET MAUI, Kotlin Multiplatform	High performance, shard codebase, strong tooling	Framework-specific learning curve, larger app size

Mapping the five approaches

UI rendering

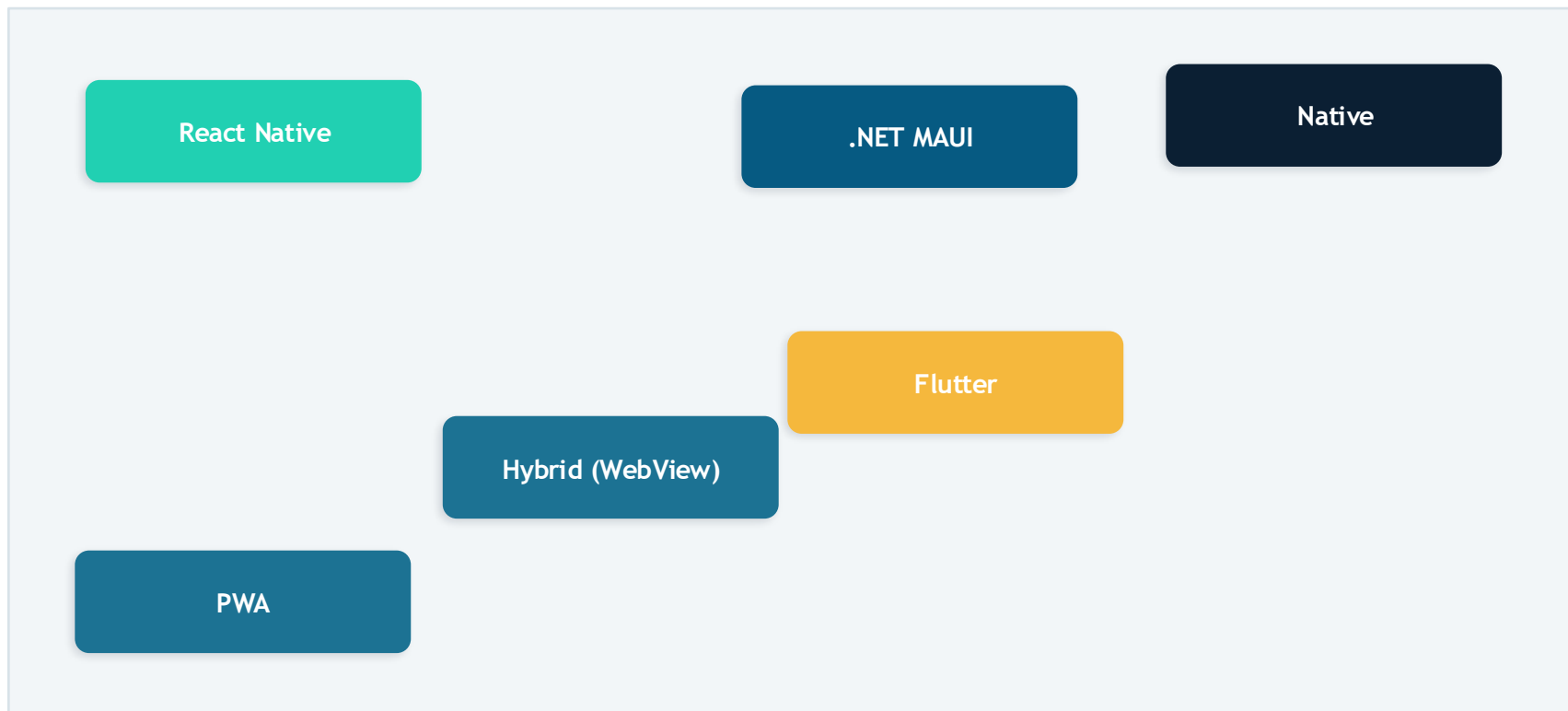
Native widgets

Flutter draws UI itself

Embedded WebView within native app shell

Web engine

Web / canvas



Native development

- **iOS: Swift + SwiftUI/UIKit (Xcode). Android: Kotlin + Jetpack Compose (Android Studio).**
- UI drawn with the platform's own widgets; logic compiled straight to machine code.



Strengths

Best performance and UX fidelity · full, day-one access to every platform API · smallest binaries



Costs

Two separate codebases and skill sets · highest cost · slowest to ship to both platforms · duplicated logic and bugs.

Hybrid architecture (WebView)

- **A web app (HTML/CSS/JS) runs inside the system WebView, wrapped in a thin native shell.**
- Native plugins bridge to device APIs. Tools: Apache Cordova (legacy), Capacitor (modern), Ionic (UI on top).



Where it fits

Content-driven apps, internal tools, or teams with a strong web codebase and simple UI needs.



Strengths

Reuse existing web code and skills · one codebase · the whole web ecosystem



Limits

WebView performance ceiling · non-native feel for complex UI/animation · reliance on plugin quality

Progressive Web Apps (PWA)

- A website with app-like powers — no native shell at all.
- Service workers add offline support and caching; a manifest makes it installable to the home screen; responsive layout adapts to any device.



Where it fits

Broad-reach content and utility apps where install friction matters more than deep device integration.



Strengths

Zero install friction · instant updates (it's just a URL) · one codebase · cheapest possible reach

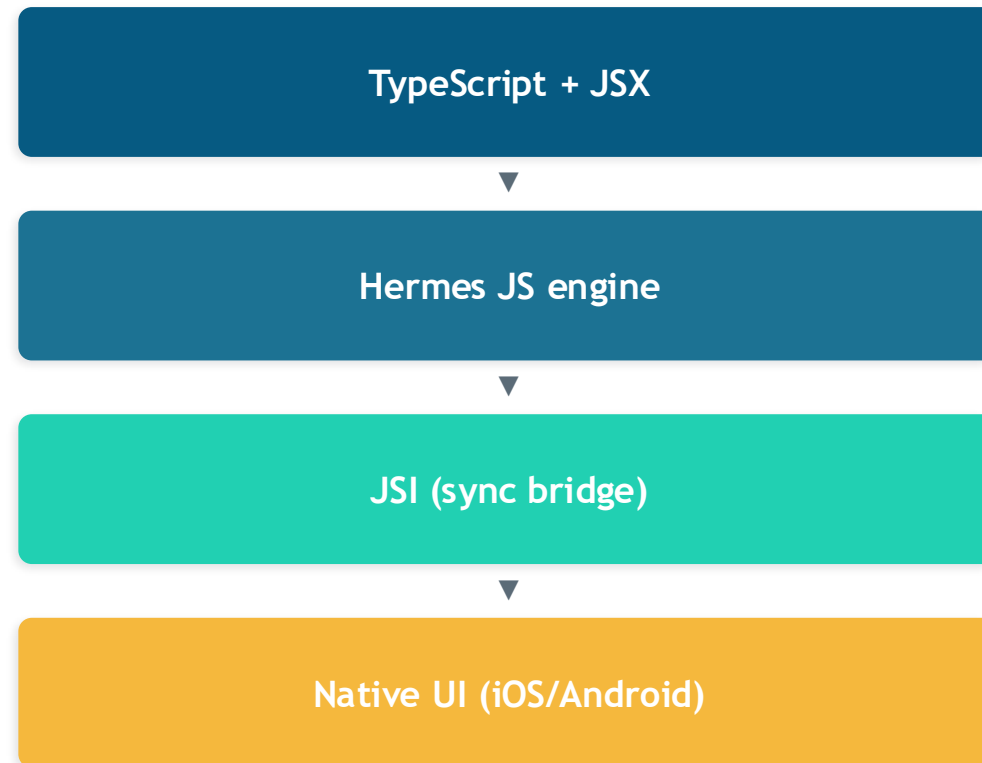


Limits

Restricted native API access (especially on iOS) · limited/no app-store presence · push and background limits · discoverability

Interpreted frameworks — React Native

- JS/TS runs in a JS engine (Hermes); the UI is real native widgets — not a **WebView**, not a canvas.
- Old model: an async, serialized 'bridge' between JS and native.
- New Architecture: JSI (a synchronous C++ interface), the Fabric renderer and TurboModules — most of the old overhead is gone.
- Also in this family: NativeScript.



Interpreted frameworks — React Native

- Your JS/TS runs in a JS engine (Hermes); the UI is real native widgets — not a WebView, not a canvas.
- Old model: an async, serialized 'bridge' between JS and native.
- New Architecture: JSI (a synchronous C++ interface), the Fabric renderer and TurboModules — most of the old overhead is gone.
- Also in this family: NativeScript.



Strengths

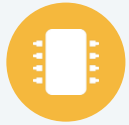
Native UI and near-native performance · one shared codebase · fast development with fast refresh · large React/expo ecosystem · access to native APIs



Limits

Native build tools required · platform-specific code may be needed · dependency and upgrade issues · complex debugging across JS and native layers

Cross-compiled frameworks — Flutter & Xamarin



Flutter (Google · Dart)

Dart is compiled AOT to native ARM code. Flutter draws its OWN UI with a rendering engine (Skia / Impeller) — it does not use platform widgets. Result: pixel-identical UI across platforms and excellent animation performance. Cost: a new language, larger binaries, UI can diverge from platform conventions.



.NET MAUI / Xamarin (Microsoft · C#)

C# compiled to native; unlike Flutter, it maps to the platform's NATIVE controls. Strong in .NET/enterprise shops. Xamarin is now succeeded by .NET MAUI. Cost: smaller web-developer overlap; ecosystem narrower than RN/Flutter.

The deep comparison matrix

Approach	Language	UI rendering	Native API	Code reuse
Native	Swift / Kotlin	OS widgets	Full, instant	None (2 apps)
Hybrid (WebView)	HTML/CSS/JS	WebView	Via plugins	High
PWA	HTML/CSS/JS	Browser engine	Limited (iOS)	Very high
React Native	JS / TS	Native widgets (JSI)	Via modules	High
Flutter	Dart	Own engine (Skia)	Via plugins	High
.NET MAUI	C#	Native widgets	Via handlers	High

React Native sits in the sweet spot for this course: native UI fidelity, high code reuse, and a JavaScript team.

What the market is doing

~70%

Android global OS share (early 2026);
iOS ~29% — together >99%

~64%

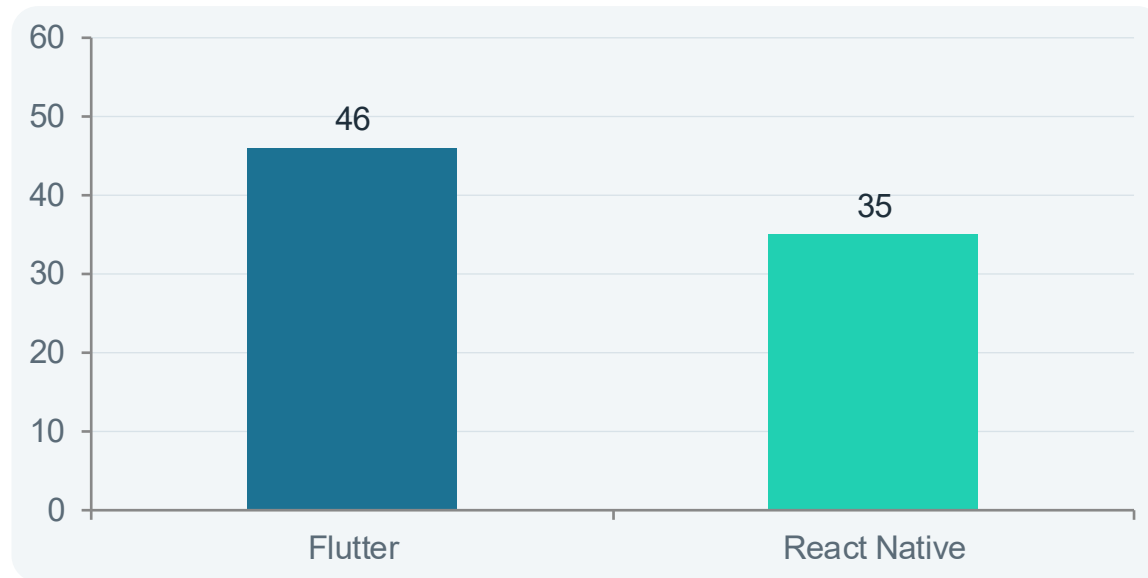
of consumer app spend is captured
on iOS

~80%

of cross-platform projects use Flutter
or React Native

8 : 1

React Native vs Flutter job listings (US,
2025)



FACT

Both Flutter and React Native are leading cross-platform; React Native has an advantage because many developers already know JavaScript, and JS ecosystem is large.

Trade-offs & decision theory

- There is no single best mobile app development approach.
- The right choice depends on the specific project and its goals.
- When selecting a tech stack, you need to balance:
 - Code reuse across platform
 - Native user experience and performance
 - Development speed and cost
- Important factors to consider include:
 - Application requirements
 - Team skills and experience
 - Available budget and timeline
 - Target platforms
 - Long-term maintenance and scalability

The best solution is the one that fits your projects needs

Max performance + deep device + platform UX

Native

Broad reach, minimal install friction

PWA

Strong web codebase, simple UI

Hybrid / WebView

Native feel + shared code + JS team

React Native

Pixel-perfect custom UI, OK with Dart

Flutter

A new Navigation

- From Session 3, we remove Drawer and related animations. Therefore the legacy Expo and React Native dependencies can be safely removed and package.json will be updated to the latest version for most dependencies.

```
"dependencies": {  
  "@expo/vector-icons": "^15.0.2",  
  "@react-native-async-storage/async-storage": "^2.2.0",  
  "expo": "~54.0.7",  
  "expo-font": "~14.0.8",  
  "expo-router": "~6.0.4",  
  "expo-splash-screen": "~31.0.10",  
  "expo-status-bar": "~3.0.8",  
  "react": "19.1.0",  
  "react-dom": "19.1.0",  
  "react-native": "0.81.4",  
  "react-native-safe-area-context": "^5.4.0",  
  "react-native-screens": "~4.16.0",  
  "react-native-toast-message": "^2.3.3",  
  "react-native-web": "~0.21.0"  
},  
"devDependencies": {  
  "@babel/core": "^7.20.0",  
  "@types/react": "~19.1.0",  
  "babel-preset-expo": "~54.0.0",  
  "typescript": "~5.9.2"  
}
```

A new Navigation

- In this new navigation structure, we have Stack Navigator at root.
- We will use Stack + Tab combined navigation throughout the rest lectures.
- `postDetails/[id].tsx` sits outside the (tabs) group. When a post is tapped, it pushes `postDetails` on top of the entire tab bar, so the tab bar disappears.
- This is different from Session 2, where the Stack was inside the Home tab and the tab bar stayed visible.

```
●●● app/ - file-based routes
app/
├─ _layout.tsx          root Stack
├─ (tabs)/              headerShown: false
│   ├─ index.tsx        Home
│   ├─ profilePage.tsx  Profile
│   └─ authenticationPage.tsx Sign In
└─ postDetails/[id].tsx pushed over tabs
```

```
export default function RootLayout() {
  return (
    <Stack>
      <Stack.Screen
        name="(tabs)"
        options={{
          headerShown: false,
          title: "Home",
        }}
      />
      <Stack.Screen name="postDetails/[id]" />
    </Stack>
    <Toast />
  </>
);
}
```

The new RootLayout

- `<Toast />` is placed in `RootLayout` so toast messages can be displayed from any screen in the application.
- In this way, you mount `<Toast />` only once. And the toast appears as an overlay above the navigation screens
- You can also mount a local `<Toast />` only in the screen or layout where you need it, i.e., `profilePage`, `authenticationPage`, etc.

```
export default function RootLayout() {  
  return (  
    <>  
      <Stack>  
        <Stack.Screen  
          name="(tabs)"  
          options={{  
            headerShown: false,  
            title: "Home",  
          }}  
        />  
        <Stack.Screen name="postDetails/[id]" />  
      </Stack>  
      <Toast />  
    </>  
  );  
}
```

State and effects

- **useState** — local, reactive data. Changing it re-renders.
 - `const [posts, setPosts] = useState<Post[]>([])`
- **useEffect** — run side-effects (fetch, subscribe, etc) after render.
 - The dependency array controls when it re-runs.
- **useFocusEffect + useCallback** — run when a screen gains focus.
 - Refresh the feed every time the user returns to it.

useFocusEffect

```
import { useFocusEffect } from 'expo-router';
import { useCallback } from 'react';

useFocusEffect(
  useCallback(() => {
    loadPosts(); // runs on focus
  }, [])
);
```

Screen Refocus

- `useEffect` takes effect only when component mounts.
- `useFocusEffect` takes effect every time the tab becomes active. `loadUser()` is called every time the tab gains focus.
- `useFocusEffect` + `AsyncStorage`: refresh data whenever user returns to the screen.
- The user registered in `asyncStorage` in Authentication Page is fetched when we navigate to Profile Page.

```
// useFocusEffect runs every time this tab gains focus.  
// This means if the user registers on the Sign In tab and then  
// taps Profile, the latest data is always shown.  
useFocusEffect(  
  useCallback(() => {  
    async function loadUser() {  
      const stored = await AsyncStorage.getItem(USER_KEY);  
      setUser(stored ? JSON.parse(stored) : null);  
      setLoading(false);  
    }  
    loadUser();  
  }, [])  
);
```

Rendering a feed with refresh control

- onRefresh + refreshing — pull-to-refresh.
- You need to import RefreshControl from react-native.
- handleRefresh() reloads posts from AsyncStorage. If storage is empty, it restores the default posts. It also shows a toast and controls the refresh indicator.
- setRefreshing controls the refresh indicator.

```
<FlatList
  data={posts}
  keyExtractor={({item}) => item.id}
  style={styles.list}
  ListHeaderComponent={() => <Spacer height={8} />}
  ListFooterComponent={() => <Spacer height={16} />}
  ItemSeparatorComponent={() => <Spacer height={12} />}
  renderItem={({ item }) => <Post postData={item} />}
  refreshControl={
    <RefreshControl refreshing={refreshing} onRefresh={handleRefresh} />
  }
/>
```

```
async function handleRefresh() {
  setRefreshing(true);
  const stored = await AsyncStorage.getItem(POSTS_KEY);

  if (stored) {
    setPosts(JSON.parse(stored));
    Toast.show({ type: "info", text1: "Posts refreshed from storage." });
  } else {
    // Storage was cleared – re-seed so the list is never empty.
    const initial = getAllPosts();
    await AsyncStorage.setItem(POSTS_KEY, JSON.stringify(initial));
    setPosts(initial);
    Toast.show({ type: "info", text1: "Storage was empty – posts re-seeded." });
  }

  setRefreshing(false);
}
```

Two Core components



ActivityIndicator

ActivityIndicator is a core React Native component, imported from react-native. It displays a loading spinner while an asynchronous operation is running.



Toast

Toast is not a core React Native component. It comes from a third-party library react-native-toast-message. It displays a short, temporary message without interrupting the user.

ActivityIndicator

- The right screen shows how ActivityIndicator works.

```
setLoading(true);
await delay(1000); // Simulate a delay

// One email can only be used once.
const existing = await findUserByEmail(email.trim());
if (existing) {
  setLoading(false);
  showError("An account with this email already exists.");
  return;
}
```

```
<TouchableOpacity
  style={[styles.button, loading && styles.buttonDisabled]}
  onPress={isSignUp ? handleSignUp : handleSignIn}
  disabled={loading}
>
  {loading ? (
    <ActivityIndicator color="white" /> // ActivityIndicator shows when loading
  ) : (
    <Text style={styles.buttonText}>{isSignUp ? "Create account" : "Sign in"}</Text>
  )}
</TouchableOpacity>
```

Toast

- The below screen shows how ActivityIndicator works.

```
// A red Toast is shown at the top when encountering any error.  
function showError(message: string) {  
  Toast.show({ type: "error", text1: message });  
}
```

```
if (password.length < 6) {  
  showError("Password must be at least 6 characters.");  
  return;  
}
```

Where to store data on the device



AsyncStorage

Simple, unencrypted key–value store. Strings only. Great for non-sensitive app data (cached feed, settings, draft posts).

Available through React Native community package

Access through simple asynchronous key-value API



SecureStore

Encrypted storage backed by the device keychain/keystore. Use for tokens, secrets, anything sensitive.

Commonly used through expo-secure-store package (you need to install the dependency)

Access through asynchronous secure key-value API

RULE

Both store strings — `JSON.stringify`, `JSON.parse`.

Install & use AsyncStorage

terminal

```
npx expo install \
  @react-native-async-storage/async-storage
```

feedback with Toast (optional)

```
import Toast from 'react-native-toast-message';
Toast.show({ type: 'success',
  text1: 'Saved', text2: 'Feed stored locally' });
```

storage helpers

```
import AsyncStorage from
  '@react-native-async-storage/async-storage';

export async function savePosts(posts: Post[]) {
  await AsyncStorage.setItem('posts',
    JSON.stringify(posts));
}

export async function loadPosts(): Promise<Post[]> {
  const raw = await AsyncStorage.getItem('posts');
  return raw ? JSON.parse(raw) : [];
}
```



Create and persist posts

YOUR TASKS

- Build a standalone screen called `CreatePostScreen.tsx` where users can:
 - Create a post using a title and description
 - Save posts to `AsyncStorage`
 - Display saved posts in `FlatList`
 - Reload saved posts when the app starts again.

DELIVERABLE

A persistent feed that can create and display posts.

EST. EFFORT

≈ 2–3 hours